



Como dois programas conversam?

Integração de Sistemas - Aula #03

Prof. Dr. Carlos Melo

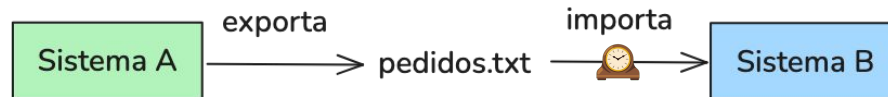


carlos.melo@upe.br



Continuando...

- Através da integração por meio de nosso parser de pedidos é possível exportar e importar arquivos entre origem e destino
- A isso chamamos de Communication Through Common Data:
 - Um serviço grava o dado em um lugar combinado e o outro lê depois
 - O formato do arquivo está descrito em contrato
- Porém, a rede é lenta e o pedido pode levar horas para ser processado, uma vez que o consumidor só descobre que tem dado novo ao conferir a pasta.





Dois programas executando separadamente

- Como que um pode pedir alguma coisa ao outro de maneira eficiente?
 - Dentro de um mesmo programa a ideia é simples

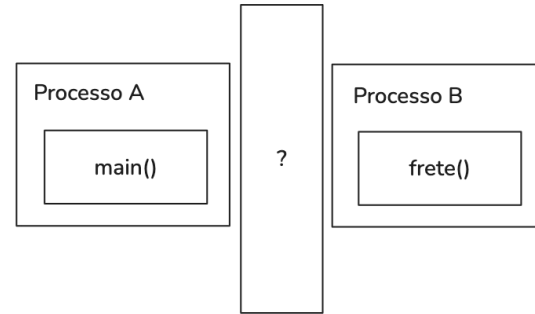
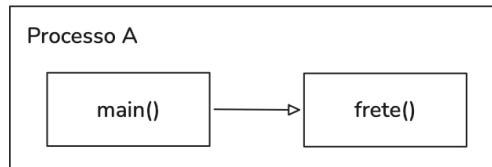
```
1 def frete(cep, peso):  
2     return peso * 4.5  
3  
4 total = frete("500000000", 2.4)
```

- main e frete moram na mesma memória
- Chamar é desviar a execução para um endereço
- Custa nanossegundos e não falha sozinho



Agora separe os dois

- Cada processo tem sua própria memória
- O sistema operacional impede que um leia a do outro
- Isso é proteção:
 - sem isso, qualquer programa leria a variável que armazena sua senha
 - **frete()** simplesmente não existe para quem está do outro lado, por exemplo



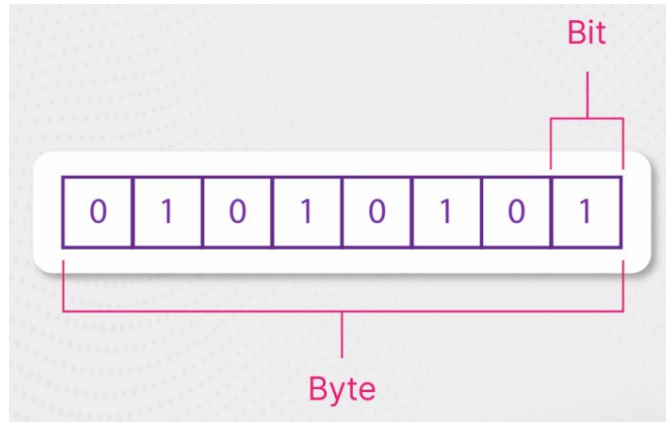


Mandando Bytes

Se não dá para compartilhar memória, resta uma coisa:

- Não vai função, não vai objeto, não vai variável
- Vai uma sequência de números de 0 a 255
- Tudo o que virá depois nesta disciplina é construído em cima disso

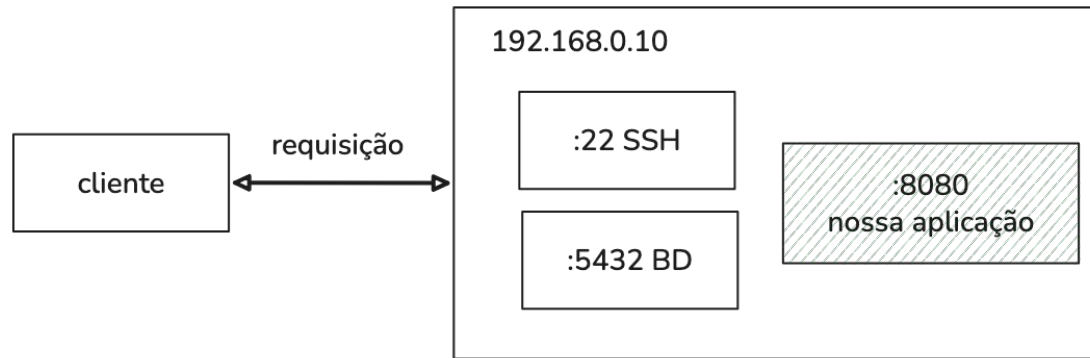
Um lado escreve bytes e o outro lê bytes





E manda pra onde?

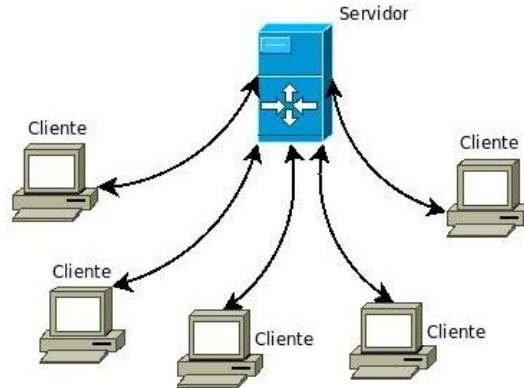
- IP: qual a máquina (dentre bilhões delas)
- Porta: Qual programa dentro daquela máquina





Sockets

- Um lado da conversa escuta numa porta e espera: o servidor
- O outro lado conecta naquele IP e porta: o cliente
- Aberta a conexão, os dois podem escrever e ler bytes
- Quem escuta e quem chama é uma decisão de projeto (e protocolo)
- Essa é a base do modelo cliente-servidor





O TCP

Garante

- Os bytes chegam na ordem enviada
- Nenhum byte chega corrompido
- O que foi perdido é reenviado

Não garante

- Que o outro lado entendeu o que recebeu
- Que o outro lado processou o que foi recebido
- Um tempo para o retorno de informação ou se ela vai voltar



Bytes sozinhos não indicam muita coisa

O servidor recebeu: 52 65 63 69 66 65 2C 32 2E 34

- Isso é um CEP e um peso? Um nome? Uma imagem?
- O TCP entregou tudo certinho, mas seria inútil sem um acordo pré-assinado
- Os dois lados precisam combinar antes o que os bytes querem dizer

Esse acordo se chama protocolo.



Um protocolo não é um programa

Um protocolo indica três pontos:

- Vocabulário: Que mensagens existem
- Formato: Como as mensagens são montadas
- Regras: Quem fala primeiro e se fala sozinho

Combinado isso, os dois lados podem ser escritos em linguagens diferentes, por times diferentes, em anos diferentes, mas o funcionamento será o mesmo.

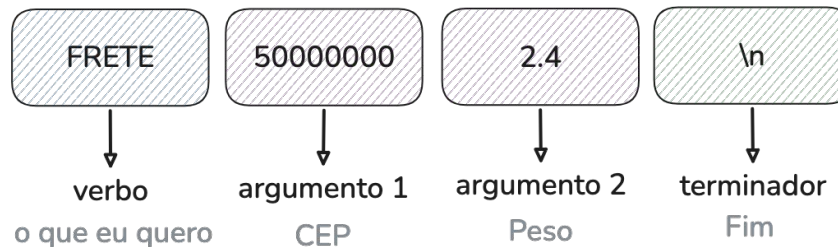


Criando nosso próprio protocolo



```
1 def frete(cep, peso):  
2     return peso * 4.5  
3  
4 total = frete("500000000", 2.4)
```

Pedido



Respostas Possíveis

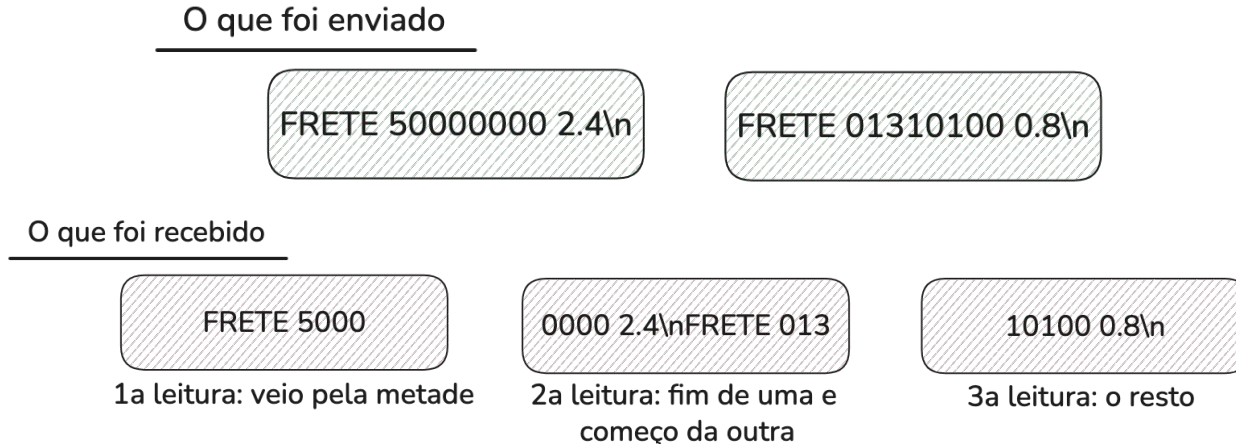
OK 10.80\n

ERR peso_invalido\n



Enquadramento

- TCP entrega fluxo contínuo e não mensagens
 - Garante ordem dos bytes, ele não sabe onde uma mensagem acaba e outra começa.
- Quem resolve é o protocolo, de um destes dois jeitos:
 - Um delimitador combinado (ex: `\n`) ou o tamanho anunciado antes do conteúdo.





Há duas formas de resolver o enquadramento

- Delimitador: combina-se um byte que nunca aparece no conteúdo.
 - Saída simples que obriga a ler a mensagem até achar esse byte
- Tamanho antes: onde os primeiros bytes dizem quantos outros virão.
 - Mais eficiente e permite alocar antes

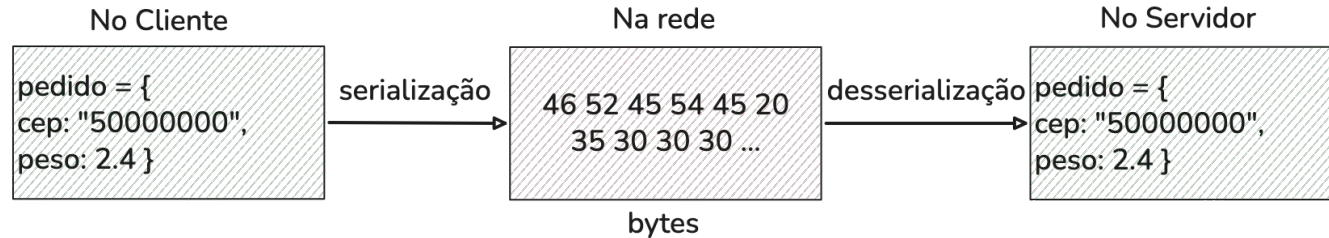
Quem usa o quê:

- Delimitador: HTTP nos cabeçalhos, SMTP e Redis
- Tamanho: HTTP no corpo (**Content-Length**), gRPC e AMQP



Serialização

- O objeto em si não atravessa, apenas a sua descrição em bytes.
- Se os dois lados discordarem do formato, B remonta errado e não recebe nenhum aviso.
- Texto é mais fácil para a gente ler, mas o binário é menor e mais rápido para o computador.





Criando um servidor em Python

```
1 import socket
2
3 def frete(peso):
4     return peso * 4.5
5
6 servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7 servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
8 servidor.bind(("0.0.0.0", 8080))
9 servidor.listen()
10
11 conexao, origem = servidor.accept()
12 buffer = ""
13 while True:
14     dados = conexao.recv(1024)
15     if not dados:
16         break
17     buffer += dados.decode("utf-8")
18     while "\n" in buffer:
19         linha, buffer = buffer.split("\n", 1)
20         verbo, cep, peso = linha.split(" ")
21         conexao.sendall(f"OK {frete(float(peso)):.2f}\n".encode())
```



Criando o cliente em Python

Nenhum framework adicional, só endereço e porta em bytes

```
1 import socket
2
3 cliente = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4 cliente.connect(("localhost", 8080))
5 cliente.sendall(b"FRETE 50000000 2.4\n")
6 print(cliente.recv(1024).decode())
```




Agora sem programa nenhum

O protocolo é texto: um humano consegue falar

```
1 $ nc localhost 8080
2 FRETE 500000000 2.4
3 OK 10.80
```



Exercício

1. Suba o servidor e chame com o **nc**
2. Acrescente o verbo **PING**, que responde **PONG**
3. Faça o servidor responder **ERR** argumentos em vez de estourar
4. Envie dois pedidos colados numa linha só e explique o que aconteceu
5. Escrevam também o cliente

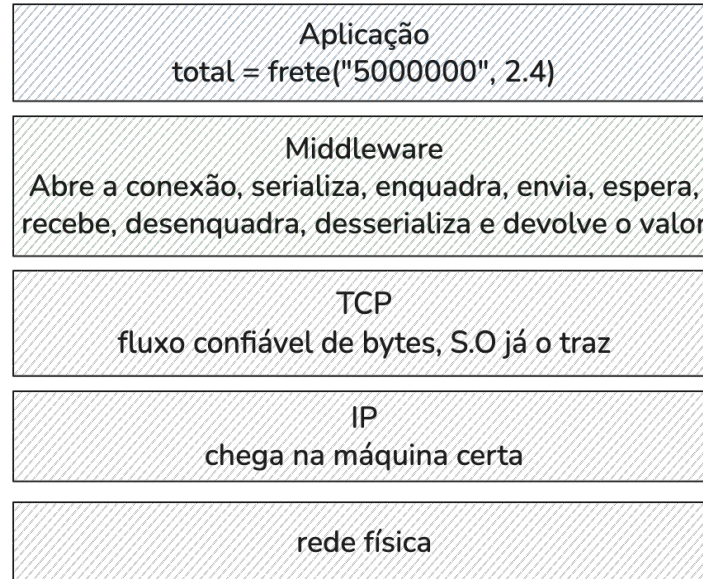


Mas... será que alguém já não fez isso antes?

- Tudo o que escrevemos é sempre igual:
 - Abrir conexão
 - Serializar
 - Enquadrar
 - Enviar
 - Esperar
 - Desserializar
- Não há mudanças significativas de projeto para projeto
- Não é o problema de negócio de ninguém
- Foi o que a indústria percebeu lá nos anos 1980'



Middleware



- Middleware é uma *camada* que fica no meio e resolve sempre a mesma coisa
- Geralmente focamos na aplicação
- **Difícilmente** você vai precisar implementar algo após o middlewa

RPC



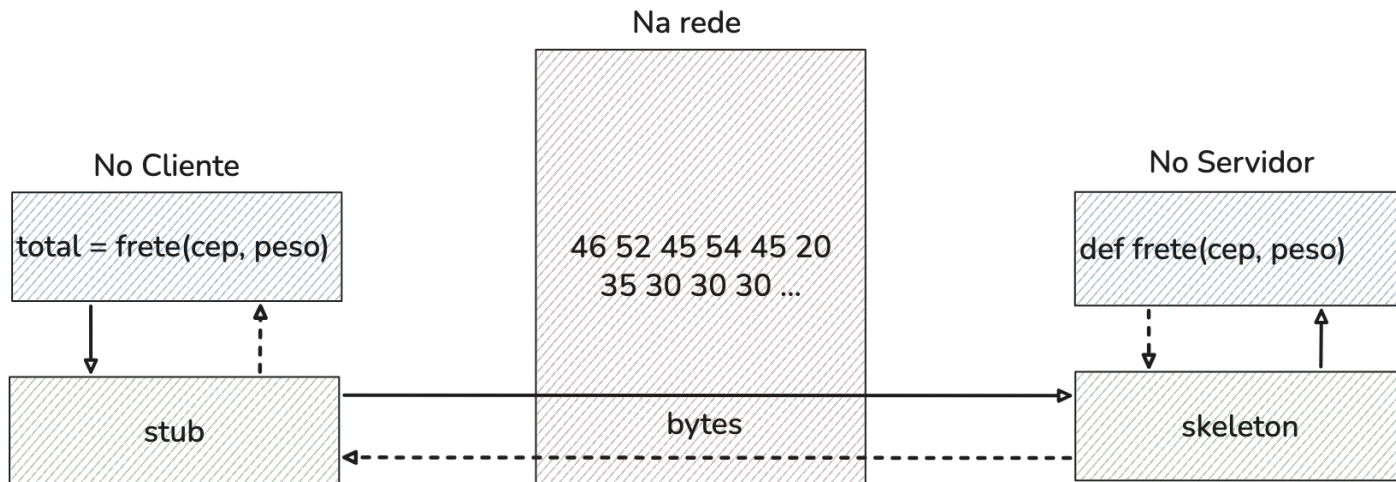
Birrell e Nelson, Xerox PARC, 1984

- E se chamar remoto fosse igual a chamar local?
 - `total = frete("50000000", 2.4)`
- Mesma linha desde o começo da aula, só que agora o cálculo de frete se dá em outro local. Porém, não há mudanças em nossa implementação.



Como o RPC funciona? Lembra lá dos dublês

- No cliente, o **stub** se passa pela função `frete()` que não está ali
- No servidor, o skeleton se passa por quem chamou, mas ninguém chamou dali
- No meio, os dois fazem serialização, envio, recebimento e desserialização
- Nenhum dos dois lados sabe que existe uma rede





Servidor e Cliente RPC



```
1 from xmlrpc.server import SimpleXMLRPCServer
2
3 def frete(cep, peso):
4     return peso * 4.5
5
6 servidor = SimpleXMLRPCServer(("0.0.0.0", 8080))
7 servidor.register_function(frete)
8 servidor.serve_forever()
```



```
1 import xmlrpc.client
2
3 servico = xmlrpc.client.ServerProxy("http://localhost:8080")
4 print(servico.frete("50000000", 2.4))
```



Simplificações

O que escrevemos	O que o RPC faz
<code>socket, bind, listen, accept</code>	<code>SimpleXMLRPCServer</code>
<code>buffer</code> e busca do <code>\n</code>	-
<code>split(" ")</code> e <code>float()</code>	-
<code>encode</code> e <code>decode</code>	-
um verbo por vez, na mão	<code>register_function</code>

- A chamada do cliente é indistinguível de uma chamada local



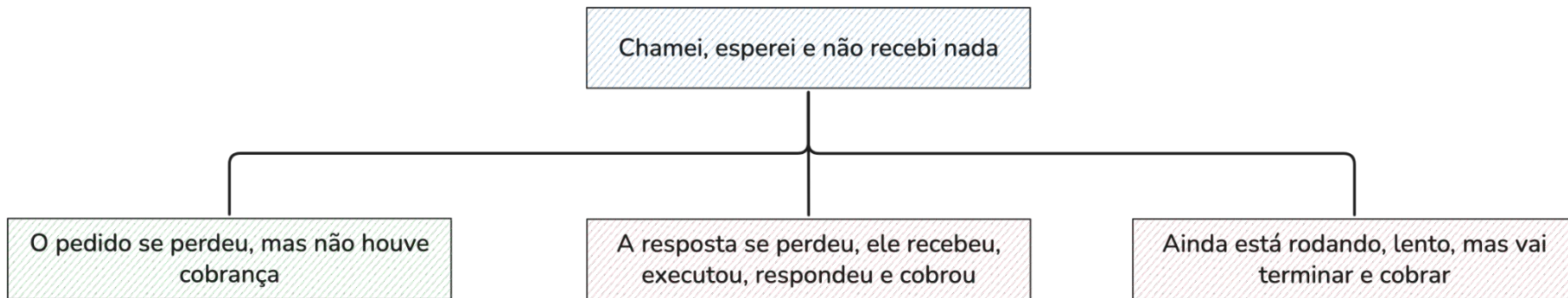
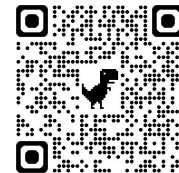
IDL (Interface Definition Language)

- Um arquivo descreve as operações, os tipos e os erros
- Uma ferramenta lê e gera os dois lados
- Cliente e servidor compilam a partir da mesma fonte

Interface Definition Language (IDL)

- Essentially a declarative language that defines types of objects by separating interfaces, named operations and parameters
- A means by which the object implementation tells clients what operations are available and how to invoke them.
- IDL is mapped to a particular language
- When compiled, it produces stubs and skeletons

Timeout ambíguo



- Os três casos são exatamente o mesmo caso.
- Decidir se vai tentar de novo ou não só cabe a você e sem saber onde você está.



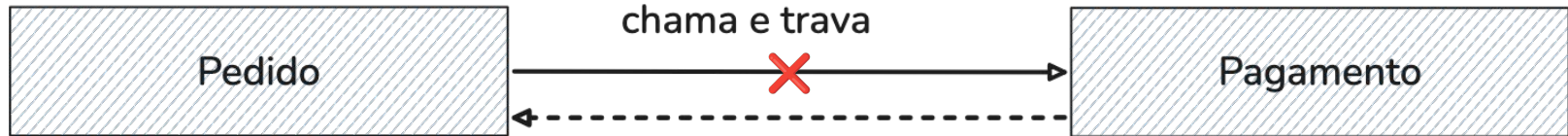
A rede não é uma chamada de função

- As falácias da computação distribuída (Peter Deutsch, 1994)
 - A rede é confiável
 - A latência é zero
 - A banda é infinita
 - A rede é segura
 - A topologia não muda
 - Há um só administrador
 - O transporte é grátis
 - A rede é homogênea
- Cada uma delas é falsa e o RPC foi construído assumindo que eram verdadeiras.



Acoplamento temporal

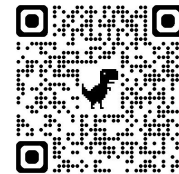
- No síncrono ambos precisam estar operacionais no mesmo instante
- E se o outro lado não estiver a disponibilidade dele virou a sua, logo há um acoplamento de tempo.





O que faltava

- Cada middleware trazia o seu próprio protocolo, incompatível com os outros.
- E cada um exigia porta própria, biblioteca própria, fornecedor próprio.
- Faltava um protocolo que todas as máquinas do mundo já falassem: HTTP.



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br